Répartition des tâches — Mini Projet WS REST

Système de Gestion Scolaire — MBDS 2025-2026

Délai : 30 mars 2026 | Travail en binôme

Structure du projet

```
src/main/java/com/example/school/
├── config/
│   ├── SecurityConfig.java
│   ├── FiltreJwt.java
│   └── ChargeurDonnees.java
├── controller/
│   ├── AuthentificationController.java
│   ├── EtudiantController.java
│   ├── ProfesseurController.java
│   ├── CoursController.java
│   ├── InscriptionController.java
│   └── NoteController.java
├── service/
│   ├── AuthentificationService.java
│   ├── JwtService.java
│   ├── EtudiantService.java
│   ├── ProfesseurService.java
│   ├── CoursService.java
│   ├── InscriptionService.java
│   └── NoteService.java
├── repository/
│   ├── UtilisateurRepository.java
│   ├── EtudiantRepository.java
│   ├── ProfesseurRepository.java
│   ├── CoursRepository.java
│   ├── InscriptionRepository.java
│   └── NoteRepository.java
├── model/
│   ├── Utilisateur.java
│   ├── Etudiant.java
│   ├── Professeur.java
│   ├── Cours.java
│   ├── Inscription.java
│   └── Note.java
├── dto/
│   ├── ConnexionDTO.java
│   ├── EtudiantDTO.java
│   ├── ProfesseurDTO.java
│   ├── CoursDTO.java
│   ├── InscriptionDTO.java
│   ├── NoteDTO.java
```

```
│   ├── ResumeEtudiantDTO.java
│   └── StatistiquesCoursDTO.java
└── exception/
    └── GestionnaireErreurs.java

src/main/resources/
├── application.properties
└── data.sql
```

Entités et leurs relations

Entité	Description	Relations
Etudiant	Un élève inscrit à l'école	S'inscrit à plusieurs cours
Professeur	Un enseignant	Enseigne plusieurs cours
Cours	Une matière enseignée	Appartient à un professeur
Inscription	Lien entre un étudiant et un cours	Relie Etudiant ↔ Cours (n à n)
Note	La note d'un étudiant dans un cours	Appartient à Etudiant + Cours

---

Rôles JWT

Rôle	Accès
`ADMIN`	Tout faire
`PROFESSEUR`	Gérer ses cours, ajouter des notes
`ETUDIANT`	Voir ses cours, ses notes, s'inscrire

Binôme 1 — Authentification + Etudiant + Note

Fichiers à créer

Sécurité et Configuration

- `config/SecurityConfig.java` — configuration Spring Security
- `config/FiltreJwt.java` — filtre qui vérifie le token JWT sur chaque requête
- `config/ChargeurDonnees.java` — insertion des données initiales au démarrage
- `exception/GestionnaireErreurs.java` — gestion globale des erreurs
- `resources/application.properties` — configuration H2, JPA, Swagger
- `resources/data.sql` — données initiales

Authentification

- `model/Utilisateur.java` — entité utilisateur avec email, mot de passe, rôle

- `repository/UtilisateurRepository.java` — recherche par email et mot de passe
- `service/JwtService.java` — génération et validation du token JWT
- `service/AuthentificationService.java` — logique de connexion
- `controller/AuthentificationController.java` — endpoint de connexion
- `dto/ConnexionDTO.java` — données de connexion (email + mot de passe)

Etudiant

- `model/Etudiant.java` — entité étudiant
- `repository/EtudiantRepository.java` — accès base de données étudiant
- `service/EtudiantService.java` — logique métier étudiant
- `controller/EtudiantController.java` — endpoints REST étudiant
- `dto/EtudiantDTO.java` — données étudiant pour les réponses
- `dto/ResumeEtudiantDTO.java` — résumé complet d'un étudiant (agrégation)

Note

- `model/Note.java` — entité note (score, commentaire, date)
- `repository/NoteRepository.java` — accès base de données note
- `service/NoteService.java` — logique métier note + calcul moyenne
- `controller/NoteController.java` — endpoints REST note
- `dto/NoteDTO.java` — données note pour les réponses

Endpoints à implémenter

```
# Authentification
POST   /api/auth/connexion              → Connexion, retourne le token JWT
POST   /api/auth/inscription            → Créer un compte étudiant

# Etudiant
GET    /api/etudiants                   → Liste tous les étudiants (ADMIN)
GET    /api/etudiants/{id}              → Détail d'un étudiant
PUT    /api/etudiants/{id}              → Modifier un étudiant
DELETE /api/etudiants/{id}              → Supprimer un étudiant (ADMIN)
GET    /api/etudiants/{id}/resume       → ⭐ Résumé complet : moyenne + liste des
cours

# Note
POST   /api/notes                       → Ajouter une note (PROFESSEUR)
PUT    /api/notes/{id}                  → Modifier une note (PROFESSEUR)
GET    /api/notes/etudiant/{etudiantId} → Toutes les notes d'un étudiant
```

Réponse agrégation — Résumé étudiant

```
{
  "id": 1,
  "nomComplet": "Jean Dupont",
```

```
  "email": "jean@school.com",
  "listeCours": ["Mathématiques", "Informatique", "Physique"],
  "moyenneGenerale": 14.5,
  "totalCredits": 9
}
```

Binôme 2 — Professeur + Cours + Inscription

Fichiers à créer

Professeur

- `model/Professeur.java` — entité professeur avec spécialité
- `repository/ProfesseurRepository.java` — accès base de données professeur
- `service/ProfesseurService.java` — logique métier professeur
- `controller/ProfesseurController.java` — endpoints REST professeur
- `dto/ProfesseurDTO.java` — données professeur pour les réponses

Cours

- `model/Cours.java` — entité cours avec nom, description, crédits, professeur
- `repository/CoursRepository.java` — accès base de données cours
- `service/CoursService.java` — logique métier cours + calcul statistiques
- `controller/CoursController.java` — endpoints REST cours
- `dto/CoursDTO.java` — données cours pour les réponses
- `dto/StatistiquesCoursDTO.java` — statistiques complètes d'un cours (agrégation)

Inscription

- `model/Inscription.java` — lien entre un étudiant et un cours (date d'inscription)
- `repository/InscriptionRepository.java` — accès base de données inscription
- `service/InscriptionService.java` — logique métier inscription
- `controller/InscriptionController.java` — endpoints REST inscription
- `dto/InscriptionDTO.java` — données inscription pour les réponses

Endpoints à implémenter

```
# Professeur
GET    /api/professeurs                          → Liste tous les professeurs
POST   /api/professeurs                          → Créer un professeur (ADMIN)
PUT    /api/professeurs/{id}                     → Modifier un professeur
DELETE /api/professeurs/{id}                     → Supprimer un professeur (ADMIN)

# Cours
GET    /api/cours                                → Liste tous les cours (PUBLIC)
POST   /api/cours                                → Créer un cours (PROFESSEUR,
ADMIN)
```

```
PUT    /api/cours/{id}                          → Modifier un cours
DELETE /api/cours/{id}                          → Supprimer un cours (ADMIN)
GET    /api/cours/{id}/statistiques             → ⭐ Statistiques : moyenne, max,
min, nb étudiants

# Inscription
POST   /api/inscriptions                        → Inscrire un étudiant à un cours
DELETE /api/inscriptions/{id}                   → Désinscrire un étudiant
GET    /api/inscriptions/etudiant/{etudiantId}  → Liste des cours d'un étudiant
```

Réponse agrégation — Statistiques cours

```
{
  "id": 1,
  "nomCours": "Informatique",
  "professeur": "Prof Martin",
  "nombreEtudiants": 25,
  "moyenneClasse": 13.2,
  "meilleureNote": 18.0,
  "noteMinimale": 7.0
}
```

Tâches communes (à faire ensemble)

Tâche	Description	Priorité
`pom.xml`	Configurer les dépendances (Web, JPA, Security, JWT, H2, Swagger)	🔴 En premier
Structure du projet	Créer les packages ensemble	🔴 En premier
HATEOAS	Ajouter les liens de navigation sur chaque réponse REST	🟡 Milieu
Tests Postman	Créer la collection Postman complète	🟡 Milieu
Documentation Swagger	Vérifier que tous les endpoints apparaissent	🟢 Fin
Revue finale	Tester l'application de bout en bout	🔴 Avant livraison

---

Planning suggéré

Jour	Binôme 1	Binôme 2
Jour 1	pom.xml + structure + modèles Utilisateur, Etudiant	Modèles Professeur, Cours, Inscription
Jour 2	Auth + JWT + Filtre sécurité	Relations entre entités

Jour	Binôme 1	Binôme 2
Jour 3	CRUD Etudiant + Note	CRUD Professeur + Cours + Inscription
Jour 4	Endpoint agrégation Résumé étudiant	Endpoint agrégation Statistiques cours
Jour 5	HATEOAS + tests	HATEOAS + tests
Jour 6	Collection Postman + Swagger + livraison	Collection Postman + Swagger + livraison

Livraison finale

- Sources complètes du projet
- Documentation Swagger accessible sur `/swagger-ui.html`
- Collection Postman exportée en `.json`
- Version en ligne si possible (ex: Railway, Render)